

1 Explication du sujet

L'idée est qu'on a un environnement, une map ($100m \times 100m$) comme dans un jeu vidéo, dans laquelle il y a des obstacles, des individus, et un drone. Le drone représente un point d'accès wifi, c'est-à-dire qu'il fournit du réseau à des utilisateurs. Le drone a au moins trois objectifs:

- **Maximiser** le débit fournit à tous les utilisateurs d'utilisateurs.
- Apprendre à **naviguer** (éviter les obstacles)
- Consommer le moins d'énergie possible (donc **Atteindre son premier objectif le plus vite possible**)

Dans notre projet, il y a donc trois composantes:

- La partie simulation (monde physique en 3D)
- La partie apprentissage
- La partie interface graphique (GUI)

Hierarchie des fichiers

- A chaque fois que je parle de simulation physique, c'est dans `simulation.c` et `simulation.h` (les `.h` sont dans `include`). Il y a aussi des fonctions qui sont liées dans `env.c`. Le module `env` est un module qui fait le lien entre la simulation physique et l'apprentissage, c'est comment le drone voit la physique de son point de vue simplifier (il exécute juste des commandes moteurs).
- Tout ce qui est lié à l'apprentissage, c'est dans `model.c` et `model.h`. C'est là que se trouve l'IA.
- Tout ce qui est lié à l'interface graphique, c'est web.
- Le reste (fonction de récompense, gestion des obstacles), c'est un peu dans `simulation` et `env`

2 Apprentissage par renforcement

Le principe du RL est très simple, l'idée c'est qu'on a un environnement, ici il est défini par la struct `world` dans `simulation.h`, c'est-à-dire que l'environnement est le monde réelle. Il a des individus, des obstacles, et des propriétés physiques (comme la gravité etc.).

```
typedef struct {
    Drone *drone;
    User *users;
    Obstacle3D *obstacles;
    int numUsers;
    int numObstacles;
    double width, height, depth;    // Dimensions de la carte.
    // Attention, depth est la vraie hauteur ! (axe Z vers le
    // haut)
} World;
```

L'agent, c'est un algorithme qui, sans connaître l'environnement, va devoir apprendre à interagir avec lui pour atteindre un objectif. A chaque instant, l'agent observe l'état de l'environnement. Dans notre simulation, l'agent observe un nouvel état toutes les **0.01 secondes**. Il va devoir choisir des actions, et en fonction de ces actions, il va recevoir des récompenses ou des pénalités. En prenant une action, l'agent arrive dans un nouvel état

Exemple: Si l'agent choisit de se déplacer vers x , il va recevoir une récompense positive s'il se rapproche d'un utilisateur, et une pénalité s'il se rapproche d'un obstacle. Il arrive dans un nouvel état où il est plus proche d'un utilisateur, ou plus proche d'un obstacle et sa position a changé. Il répète ce processus, et au fil du temps, il apprend à choisir les actions qui lui rapportent le plus de récompenses, c'est-à-dire les actions qui l'aident à atteindre son objectif. Il cherche donc à **maximiser ses récompenses** (c'est le principe le plus important de tout le projet).

Dans toute la suite, on appelle

- $R(s, a)$ ou plus simplement R la fonction de récompense. C'est la récompense obtenue à l'instant t en prenant l'action a dans l'état s .
- $T(s, a, s')$ ou plus simplement T la fonction de transition. C'est l'état s' dans lequel on arrive à l'instant $t + 1$ en prenant l'action a dans l'état s .
- On retiendra $T(s, a, s', r)$. On associe aussi la récompense obtenue en faisant cette transition.
- S l'ensemble des états
- A l'ensemble des actions

Remarque: R et T sont des fonctions, mais dans la pratique, on va représenter R par un **double**, c'est la somme cumulée des récompenses obtenues en prenant l'action a dans l'état s . Pourquoi ? Car on peut avoir une pénalité de temps (si on va trop lentement), une pénalité de collision (si on se rapproche d'un obstacle), une récompense de proximité (si on se rapproche d'un utilisateur), etc. Donc R est la somme de toutes ces récompenses et pénalités. T c'est un buffer qui contient les données associées.

On a dit que l'objectif **ultime** c'est de maximiser les récompenses. Rien de plus simple, il faut maximiser le résultat de cette équation (**équation de Bellman**):

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') \max_{a'} Q(s', a')$$

$Q(s, a)$, c'est ce qu'on appelle une Q-value, c'est la valeur de l'action a dans l'état s . C'est donc un tableau qui contient la valeur de chaque action dans chaque état. Attention, **ce n'est pas** la récompense obtenue en prenant l'action a dans l'état s , c'est la somme de la récompense immédiate $R(s, a)$ et de la récompense future $\gamma \sum_{s'} T(s, a, s') \max_{a'} Q(s', a')$. En gros, c'est une estimation temporelle de la récompense.

La somme $\sum_{s'} T(s, a, s') \max_{a'} Q(s', a')$ représente la récompense future, c'est-à-dire la récompense qu'on peut espérer obtenir en prenant l'action a dans l'état s **immédiatement** ($T(s, a, s')$) et en suivant la meilleure politique par la suite (c'est à dire en choisissant les actions qui maximisent $Q(s', a')$ dans les états futurs s').

Mais comment on obtient $Q(s, a)$ puisque qu'on a pas encore simulé le futur ? C'est là que le **Q-learning** entre en jeu. En fait, au début de l'apprentissage, on initialise $Q(s, a)$ à 0 pour tous les états s et toutes les actions a . La première fois qu'il fera le calcul, ce sera $Q(s, a) = R(s, a)$ donc la récompense immédiate. Ensuite, ce sera $R + R'$, puis $R + R' + R''$, etc. Donc au fil du temps, $Q(s, a)$ va converger vers la valeur réelle de l'action a dans l'état s . Il faut le voir un peu comme un effet "récurive" ou de "backpropagation" dans les réseaux de neurones.

On finit par l'hyperparamètre γ . Appelé le **facteur de discount**, il permet de donner plus ou moins d'importance aux récompenses futures par rapport aux récompenses immédiates. On remarque que si $\gamma = 0$, l'équation devient $Q(s, a) = R(s, a)$, c'est-à-dire que l'agent ne se soucie que des récompenses immédiates. Si $\gamma = 1$, l'agent se soucie autant des récompenses futures que des récompenses immédiates. Nous, on prendra $\gamma = 0.99$ car le problème est très complexe. Comme on le verra après, on va aussi utiliser un réseau de neurone et ce qui compte c'est juste le résultat tout à la fin.

Algo du Q-learning

Une notion **très importante** à comprendre, c'est la notion d'**exploration vs exploitation**. L'idée c'est que l'agent doit trouver un équilibre entre l'exploration de nouvelles actions et l'exploitation des actions qu'il connaît déjà. Si l'agent exploite toujours les actions qu'il connaît déjà, il risque de se retrouver coincé dans une solution sous-optimale (comme un mauvais minimum local en DL). Si

l'agent explore toujours de nouvelles actions, il risque de ne jamais converger vers une solution optimale.

```
QLearning <- function(states, actions, T, R, max_epochs = 100,
  max_step = 10, epsilon = 0.1) {

  // Initialisation de la table Q
  Q <- [[0]]

  // On sauvegarde l'etat initial
  initial_state <- states[1]

  // Les epochs, c'est le nombre de fois qu'on va faire
  // l'apprentissage complet, cad le nombre de fois qu'on va
  // faire le processus d'apprentissage du debut a la fin.
  for (epoch in 1:max_epochs) {
    state <- initial_state // A chaque epoch, on recommence a
    // l'etat initial

    // Le steps, c'est le nombre d'actions et d'etat que
    // l'agent va vivre dans la simulation physique pendant
    // une epoch.
    for (step in 1:max_step) {

      // On tire un nombre aleatoire pour savoir si on va
      // explorer ou exploiter
      if (runif(1) < epsilon) {
        action <- rand(nbaction) // On explore
      } else {
        action <- which.max(Q[state, ]) // On exploite, on
        // prend l'action qui a la meilleure Q-value dans
        // l'etat actuel
      }

      new_state <- T(state, action) // C'est un step de la
      // simulation physique, donc de l'environnement, appele
      // envStep dans le code
      reward <- R(state, action) // getReward

      // Mise a jour de la table Q selon l'equation de Bellman
      Q[state, action] <- Q[state, action] + (reward + gamma *
        max(Q[new_state, ]))

      state <- new_state
    }
  }
  return (Q)
}
```

3 Q Network

Le problème du Q-learning, c'est que si on a un grand nombre d'états et d'actions, la table Q devient très grande et difficile à gérer. Et c'est le cas

ici car en fait, il y a un nombre infini d'états, car les positions sont des nombres flottants, mais on définit aussi l'état du drone par sa vitesse, son orientation, sa vitesse angulaire... et il doit prédire en fonction de ça

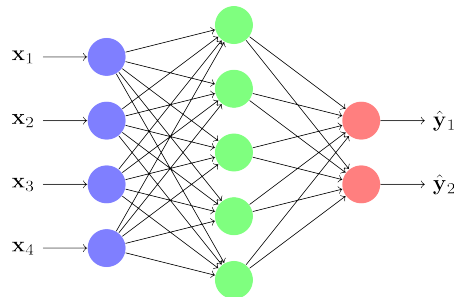
```
typedef struct {
    double x, y, z; // Position
    double x_dot, y_dot, z_dot;
    double x_dot_dot, y_dot_dot, z_dot_dot;
    double phi, theta, psi;
    double phi_dot, theta_dot, psi_dot;
    double p, q, r; // Vitesse angulaire
    double p_dot, q_dot, r_dot;
    double omega[4]; // Vitesses de rotation des moteurs
    ...
} Drone;
```

C'est pour cette raison qu'on va utiliser un réseau de neurone pour approximer la fonction Q , c'est ce qu'on appelle le **Deep Q-learning**. Donc on va remplacer Q par QN (pour Q-Network), qui est un réseau de neurone qui prend en entrée le vecteur d'état et qui sort un vecteur de Q-values pour chaque action possible:

$$QN(s) = [Q(s, a_1), Q(s, a_2), \dots, Q(s, a_n)]$$

Du coup, si on veut savoir quelle action prendre, on fait un **argmax** sur ce vecteur de sortie.

La déso ça va devenir un peu plus compliqué mais j'ai pas de tableau pour faire les dessins: les X , c'est S , le vecteur d'états, et les Y , c'est $QN(s)$, le vecteur de Q-values pour chaque action.



Là on a fait le plus dur, tout le reste c'est des extensions et des détails d'implémentation, pour résoudre tous les **problèmes** qui vont s'accumuler à cause de la nature du problème (espace continu).

4 Target Network

On réécrit l'équation de Bellman pour le Q-Network:

$$QN(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') \max_{a'} QN(s', a')$$

Et là on regarde l'image au-dessus et on se rappelle que quand un réseau de neurone s'entraîne, on fait pour chaque epoch une descente de gradient:

- Je fais une forward pass pour calculer $QN(s, a)$ (prédiction)
- Je calcule l'erreur entre $QN(s, a)$ et $R(s, a) + \gamma \sum_{s'} T(s, a, s') \max_{a'} QN(s', a')$ (Rappel si vous avez pas encore bien compris revenez plus haut mais ce membre temps vers la valeur optimale de l'action), c'est équivalent à $y - y'$ dans la descente de gradient classique.
- Je fais une backward pass pour mettre à jour les poids du réseau de neurone

Et là y'a un putain de problème qui va nous traîner tout du long, c'est qu'une backpropagation modifie les poids, donc elle modifie la fonction QN elle-même, et du coup, QN' change pour la prochaine itération, ça veut dire que l'action optimale pour l'état s' change à chaque itération, et du coup, on peu pas converger. C'est comme si notre objectif en athlétisme c'est d'atteindre la ligne d'arrivée, mais à chaque fois que je fais un pas, la ligne d'arrivée se déplace de la même distance (car le gradient applique la même valeur dans la même direction), c'est impossible de l'atteindre.

Pour régler ce problème, on utilise une technique appelée **Target Network**. On va utiliser deux réseaux de neurones, QN et QT' . QN est le réseau de neurone principal qui est mis à jour à chaque itération, et QT est un réseau de neurone "cible" qui est une copie de QN mais qui n'est mis à jour que tous les C épisodes (par exemple tous les 10 épisodes). En gros, notre cible reste fixe pendant un certain temps, du coup on sait où aller et comment l'atteindre, puis ensuite elle bouge, mais sur le long terme on la "rattrape" car on a le temps de s'adapter. Dans le code, on synchronise tous les `networks_update_freq`.

Equation **finale** à résoudre:

$$QN(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') \max_{a'} QT(s', a')$$

5 Replay Buffer

Maintenant, un autre problème du continu, c'est que dans le Q-Learning classique, on choisit une action, on observe le nouvel état, et on met à jour la table. Ici si on fait ça, on va avoir un problème de corrélation des données.

Exemple: On a dit que chaque pas de la simulation dans le monde physique c'est $DT = 0.01s$. Si à l'instant t , le drone est en $(x = 10, y = 10)$, à l'instant $t + 0.01$, il est en $(x = 10.1, y = 10)$. Ces deux états sont quasi identiques. Si on entraîne le réseau dessus, il va overfitter sur cette trajectoire et ne pas apprendre à voler. En fait ça peut être pire, la différence dans le calcul de l'erreur (gradient descent) peut être tellement faible qu'on a du *vanishing gradient*. Tous les poids vont passer à 0, et ce à cause de la corrélation des données.

Ici la solution, c'est d'utiliser un **Replay Buffer**. On stocke les transitions, c'est-à-dire les tuples (S, A, R, S') dans un buffer de taille fixe (par exemple 25 000). A chaque fois que l'agent fait une action, on stocke la transition dans ce buffer. A chaque fois qu'on veut prédire, on ne prend pas la transition la plus récente, mais on tire aléatoirement un mini-batch (ex: 64) dans ce buffer pour que le réseau apprenne sur un mélange de situations passées.

Let's go maintenant on a ça:

```

typedef struct {
    double state[NB_STATES]; // Etat avant l'action
    EngineAction action; // Action qu'on prend
    double reward; // Recompense qu'on obtient
    double next_state[NB_STATES]; // Prochaine etat dans lequel
        on se trouve
    int next_state_terminal; // Est-ce que le prochain
        etat est terminal (voir Env) ?
    int flag; // Utilise pour savoir si une
        donnee est deja tiree pour un batch (voir ReplayBuffer)
    double td_error; // Utilise pour modifier les
        probabilites des tirages aleatoire (ReplayBuffer)
} Transition;

/*
 * Le ReplayBuffer est la structure qui permet de stocker
 * toutes les transitions (voir ci-dessus)
 */

typedef struct {
    Transition *buffer; // Tableau alloue dynamiquement pour
        memoriser les transitions
    int capacity; // Capacite max du buffer
    int size; // Taille actuelle du buffer
    int head; // Index d'insertion (buffer dynamique)
} ReplayBuffer;

typedef struct s_rl_model {
    NeuralNetwork *q_network; // Reseau principa
    NeuralNetwork *target_network; // Reseau cible
    Env *env; // Environnement
    ReplayBuffer *memory; // Memoire pour
        l'Experience Replay
    Config config; // Config avec
        hyperparametres

    int batchSize; // Taille des batchs pour
        les reseaux de neurones
    double learningRate; // Pour l'entrainement du
        reseau de neurone
    double decay; // Decay pour le learning
        rate du reseau de neurone
    int networks_update_freq; // Frequence de
        synchronisation des deux reseaux (ex: 1000 steps)
    int step_count; // Compteur pour savoir
        quand synchroniser
    char *path; // Chemin de sauvegarde

    ...
} DQNModel;

```

6 DeepQLearning et updateNetworks

On est aussi capable de comprendre l’algo du DQN (aller voir direct dans `model.c`) Mais là y’a un autre problème, c’est que quand on entraîne un réseau de neurones classique, on utilise des `inputs` et des `expecteds_outputs` (pour calculer l’erreur, il faut savoir ce qu’on cherche).

Cette partie est un peu chiant mais bien rodée dans le code, si ça vous fait déjà chier, vous pouvez passer. C’est sur la construction de ce vecteur.

Etapas (si besoin de plus de détails, allez voir le code):

- On tire un mini-batch de transitions dans le replay buffer $\{(s, a, r, s')\}$.
- Une première prédiction avec le réseau de neurone principal QN pour tous les états s du mini-batch, nous donne un vecteur de Q-values pour chaque état: $QN(s) = [Q(s, a_1), Q(s, a_2), \dots, Q(s, a_n)]$.
- Une deuxième prédiction avec le réseau de neurone cible QT pour tous les états s' du mini-batch, nous donne un vecteur de Q-values pour chaque état futur: $QT(s') = [Q(s', a_1), Q(s', a_2), \dots, Q(s', a_n)]$.
- On construit le vecteur d’outputs attendus pour chaque transition du mini-batch. Pour chaque transition (s, a, r, s') , on calcule la cible de la manière suivante:
 - Si s' est un état terminal, la cible est simplement r
 - Sinon, la cible est $r + \gamma \max_{a'} QT(s', a')$
- On met à jour la valeur de $QN(s, a)$ (donc de l’action réellement choisie) dans le vecteur d’outputs attendus avec la ”nouvelle” meilleure action qui doit la remplacer, tandis que les autres valeurs de $QN(s, a')$ pour $a' \neq a$ **ne doivent pas** changer (puisque le drone à choisi qu’un action, faut pas effacer sa connaissance des autres !)

7 Le Frame Skipping

Toujours le même problème (même si on a réglé les problèmes de corrélation avec le replay buffer, on a toujours un problème de corrélation temporelle).

Résumé hyper rapide ici au cas où vous avez le cerveau complètement baisé,

- On veut maximiser les récompenses du drone avec une équation théorique (que j’ai pas inventé)
- Comme on a un espace continue, on prédit l’action qu’on va prendre avec un réseau de neurones

- A cause de la mise à jours des poids et des gradients, on doit utiliser un deuxième réseau qui sert d'objectif temporaire à atteindre et permettre la convergence
- Mais a cause de la nature du problème, on a des problèmes de corrélation de données (effects très similaires sur un instant très petit quelque soit l'action), donc on utilise un replay buffer pour mélanger les données.
- Mais on a toujours un problème de corrélation temporelle...

Dans notre simulation, on a dit que l'agent observe un nouvel état toutes les 0.01 secondes, c'est à dire que le drone fait une action toutes les 0.01 secondes. Mais en fait, au bout de 0.01 secondes, le drone ne voit pas encore les effets de son action. Car ici pour rappel je fais une vrai simu physique (pour me faire chier) dans la force de poussée des moteurs par exemple à une certaine durée réelle, et il faut un certain temps pour que le drone atteigne sa futur position avec la vitesse et l'orientation associée. En gros, le temps est discrétisé mais pas les actions (continu total).

Pour régler ce problème, on utilise une technique appelée **Frame Skipping**. Au lieu de faire une action à chaque pas de temps (0.01s), on va faire une action tous les k pas de temps (par exemple tous les 4 pas de temps, soit tous les 0.04s). Pendant les pas de temps intermédiaires, on va répéter la même action, et on va accumuler les récompenses obtenues pendant ces pas de temps intermédiaires. Du coup, on réduit la corrélation temporelle, et on permet au drone de voir les effets de son action avant de devoir en choisir une nouvelle.

Là je mets l'algo car il est tout petit et c'est juste dans la fonction de transition (env)

```

void envStep(Env *env, double *next_state, double *reward, int
*is_terminal, int action_idx) {
    World *w = env->physical_world;
    double accumulated_reward = 0.0;
    int crashed = 0;

    handleCommand(w->drone, action_idx);

    // On laisse le drone executer l'action choisie pendant
    FRAME_SKIP iterations physiques
    for (int i = 0; i < FRAME_SKIP; i++) {
        physicsStep(w, DT);
        crashed = isDroneCrashed(w);
        accumulated_reward += getReward(env);
        if (crashed) break;
    }

    getStateVector(env, next_state);

    double final_reward = accumulated_reward / FRAME_SKIP;

    if (crashed) { final_reward -= 100.0; }
    *reward = final_reward;
    *is_terminal = crashed;
}

```

8 Grid Density Mapping

Là, c'est une notion importante pour la fonction de récompense, mais aussi pour la construction de l'état. On revient un peu sur l'architecture d'un réseau de neurones.

En gros, ça paraît évident (du coup vous vous êtes peut-être pas encore posé la question), pour faire une prédiction, le réseau de neurone doit prendre en entrée un vecteur de **taille fixe**, et il doit sortir un vecteur de taille fixe. Le vecteur de taille fixe c'est bon on l'a, c'est ça (les actions):

```

typedef enum {
    ENGINE_IDLE = 0,
    ENGINE_UP,
    ENGINE_DOWN,
    ENGINE_PITCH_LEFT,
    ENGINE_PITCH_RIGHT,
    ENGINE_ROLL_LEFT,
    ENGINE_ROLL_RIGHT,
    ENGINE_YAW_LEFT,
    ENGINE_YAW_RIGHT
} EngineAction;

```

Par contre pour les états, c'est plus compliqué, car l'état du drone est défini

par sa position, sa vitesse, son orientation, etc. Mais il y a aussi les utilisateurs, et les obstacles, et il peut y en avoir un nombre variable, et ils peuvent être à des positions différentes à chaque instant. Donc on peut pas juste faire un vecteur d'état qui contient la position de chaque utilisateur et de chaque obstacle, car ce vecteur aurait une taille variable, et il serait aussi très grand car si on a 100 utilisateurs, on aurait un vecteur de 300 dimensions juste pour les utilisateurs (x, y, z pour chaque utilisateur), et pareil pour les obstacles.

Par exemple: dans les prochaines choses à implémenter (le travail restant est à la toute fin), on doit mieux gérer la génération aléatoire des obstacles et des utilisateurs pour que l'agent puisse bien généraliser. Mais faut pas modifier à chaque fois les buffers en dur dans les `struct`.

On veut **1.** savoir combien y'a d'utilisateurs dans la map, **2.** savoir où ils sont, et **3.** faire tout ça avec un vecteur de taille fixe. Compter les utilisateurs, OK, on pourrait prendre un seul entier qui contient le nombre d'utilisateurs, mais ça nous dit pas où ils sont, et c'est pas suffisant pour la fonction de récompense. Autrement dit, le réseau ne peut pas faire la corrélation entre le nombre stocké dans cette variable d'état et la position des utilisateurs, et du coup il ne peut pas apprendre à se rapprocher d'eux pour maximiser le débit.

Par contre, si on prend **plusieurs** compteurs, magiquement, le réseau va se débrouiller. En fait c'est simple, avoir plusieurs compteurs, c'est avoir un vecteur:

$$\text{state} = \begin{bmatrix} \text{cpt_1} \\ \text{cpt_2} \\ \vdots \\ \text{cpt_N} \end{bmatrix}$$

C'est aussi avoir une matrice où on a nos N compteurs répartis en k lignes et k colonnes:

$$\text{state} = \begin{bmatrix} \text{cpt}_1 & \text{cpt}_2 & \cdots & \text{cpt}_k \\ \text{cpt}_{k+1} & \text{cpt}_{k+2} & \cdots & \text{cpt}_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ \text{cpt}_{(k-1)k+1} & \text{cpt}_{(k-1)k+2} & \cdots & \text{cpt}_{k^2} \end{bmatrix}$$

(Matrice faite par l'IA mdr)

Et donc ça revient à diviser notre map en une grille de $k \times k$ cellules, et chaque compteur va compter le nombre d'utilisateurs dans sa cellule. C'est le principe de **Distribution Spatiale**. Nous on a `GRID_SIZE` qui vaut 4, et donc on a 16 cadrants. Chaque case c'est comme un pixel d'une image basse résolution, et la valeur de ce pixel c'est le nombre d'utilisateurs dans cette case.

Au final, le réseau va recevoir, si on a 4 utilisateurs, quelque chose comme ça (une mini-carte thermique ou *Heatmap*)

$$\text{state} = \begin{bmatrix} 0.0 & 0.0 & 0.0 & 0.0 \\ 0.25 & 0.0 & 0.25 & 0.0 \\ 0.0 & 0.25 & 0.0 & 0.25 \\ 0.0 & 0.0 & 0.0 & 0.0 \end{bmatrix}$$

Chaque valeur est normalisée par le nombre total d'utilisateurs, donc elle est comprise entre 0 et 1, et elle représente la densité d'utilisateurs dans cette cellule. Et le réseau voit immédiatement où sont les zones de forte concentration humaine. Il faut **pas une trop grosse grille** pour pas rajouter trop d'états, sinon faut rajouter des couches de neurones, et c'est plus lent.

Du coup l'algo est le suivant, on suppose que le drone sait où sont les utilisateurs. C'est pas déconnant, car dans la réalité, un drone peut détecter les utilisateurs grâce à leur signal wifi, et il peut estimer leur position grâce à la force du signal. Ce qu'il fait c'est qu'il itère sur chaque *user* en regardant sa position, il calcule quel est l'**index** de la cellule (c'est la position de l'utilisateur divisée par la taille d'une cellule) et il incrémente le compteur de la cellule correspondante. Il place cette valeur dans le tableaux (qui sera le vecteur d'entrée du réseau).

```
static void fillUsersGrid(Env *env, double *state_out) {
    World *w = env->physical_world;

    int num_cells = GRID_SIZE * GRID_SIZE;
    for (int i = 0; i < num_cells; i++) state_out[i] = 0.0;

    for (int i = 0; i < w->numUsers; i++) {
        int cell_x = (int)(w->users[i].x / (w->width /
            GRID_SIZE));
        int cell_y = (int)(w->users[i].y / (w->height /
            GRID_SIZE));

        if (cell_x < 0) {cell_x = 0;} if (cell_x >= GRID_SIZE)
            {cell_x = GRID_SIZE - 1;}
        if (cell_y < 0) {cell_y = 0;} if (cell_y >= GRID_SIZE)
            {cell_y = GRID_SIZE - 1;}

        state_out[cell_y * GRID_SIZE + cell_x] += 1.0;
    }

    for (int i = 0; i < num_cells; i++) {
        if (w->numUsers > 0) state_out[i] /= w->numUsers;
    }
}
```

9 Capteur LiDAR pour les obstacles

Pour les obstacles, on peut pas supposer connaître leur position, car dans la réalité, un drone doit détecter les obstacles grâce à des capteurs comme le LiDAR. Il y a beaucoup d'arbres quand même ! Le LiDAR ça envoie des rayons dans toutes les directions, et il mesure le temps que mettent ces rayons à revenir après avoir touché un obstacle, ce qui lui permet de construire une carte de son environnement immédiat. Du coup on suppose qu'on détecte est capable de détecter les 3 obstacles les plus proches (tri à bulle) et on met leur position dans le vecteur d'état, après les compteurs de la grille des utilisateurs.

Tout ça là, c'est dans `env.c`

10 Curriculum Learning

Là c'est ultra simple. L'idée du curriculum learning, c'est de faire apprendre à l'agent sur des tâches de plus en plus difficiles, pour lui permettre de mieux généraliser. Du coup, quand on réinitialise l'environnement au début de chaque *epoch* (si besoin revoir l'algo du Q-learning plus haut), on peut juste randomiser la position des utilisateurs et des obstacles en fonction du nombre d'*epoch*. Donc par exemple 0 -> 200, même environnement, 200 -> 500, randomisation légère et 500+, randomisation totale. Attention, faut quand même gérer les collisions avec les bords de map, etc. j'en reparle dans la partie "Travail restant".

11 Priority Experience Replay

Là c'est aussi simple mais c'est plus lié à l'expérimentation. En gros on remarque facilement quand on lance l'entraînement que certaines transitions sont plus **importantes** que d'autres pour l'apprentissage, car elles contiennent des informations plus "riches". Par exemple de tant en tend, le drone va récupérer +1000 de récompense en se rapprochant d'un utilisateur, mais comme on a pas beaucoup de **trajectoires**, ou plutôt **d'états** optimaux (car les trajectoires on peut quand même arriver dans l'état de plusieurs manière), il va pas en voir beaucoup, et du coup il va pas apprendre à les atteindre.

Petite remarque, on pourrait croire que le problème est très complexe et qu'il n'y a pas de solution unique. En fait, la solution est difficile à choper à cause de tous les problèmes qu'on a vu. Mais une fois que c'est réglé, tout se débloque magiquement et le drone apprend à voler et à se rapprocher des utilisateurs pour maximiser le débit. En fait, le calcul de la position optimale (y aller le plus rapidement possible, ne pas toucher d'objets, maximiser le débit pendant un certain temps) est déterministe. Si on a un très bon réseau, on pourra avoir à chaque fois le même résultat et les mêmes poids dans le fichier

même si c’est les batchs sont random. Là je suis pas sûr à 100% mais en gros c’est ça (notamment pour la vision).

Du coup, pour régler ce problème, on peut utiliser une technique appelée **Priority Experience Replay**. Au lieu de tirer les transitions du replay buffer de manière complètement aléatoire, on peut leur attribuer une priorité en fonction de leur ”importance” pour l’apprentissage, et tirer les transitions en fonction de cette priorité. Par exemple, on peut donner une priorité plus élevée aux transitions qui ont une grande erreur de prédiction (c’est à dire une grande différence entre la récompense obtenue et la récompense prédite par le réseau), que ce soit une récompense très positive ou très négative car ce sont ces transitions qui vont permettre au réseau de corriger ses erreurs et d’apprendre.

Normalement c’est hyper dur à implémenter, du coup la version simplifier tire seulement 2 de ces transition, et prend celle avec la plus grande différence. Ça à pas l’air ouf mais pour les axes d’amélioration, y’a pas à aller voir trop de ce côté, là c’est un peu du **overkill**. Pour le jeu d’échec par exemple, *DeepMind* ne l’utilisait même pas alors que c’est 15x plus compliqué. Mais c’est la qu’on se rend compte que le pb est compliqué à cause du fait que je le fais en C (le continue était **imposé**) car par contre toutes les autres technique dont j’ai parlé, elles sont **obligatoires** car elles corrigent des comportements critiques.

12 Travail Restant

Malheureusement, et ben malgré tout ça, ça marche pas hyper bien. Déjà le code est un peu dégueu. Alors y’a pas besoin de regarder tous les fichiers, notamment ce qui est réseau de neurones, simulation un tout petit peu (ce qui est sur les obstacles) et la partie ”model” (DQNModel) est assez complète, toucher aux algo casserait tout.

Le principal, c’est dans env, pour la fonction de récompense et la gestion des obstacles, du signal, ... et aussi les paramètres (là y’en a dans **config** et un peu partout par contre). En fait, d’un point de vue entraînement, il peut y avoir 3 grandes sources de problèmes:

- La **fonction de récompense**, le changement **d’une valeur** peut faire passer le réseau de 53% d’accuracy ou 94% d’accuracy. C’est hyper sensible car toutes les mises à jour se basent dessus.
- Le **comportement physique**. Le dernier changement qui m’a fait grimper la récompense et qui fait que c’est mieux, c’est d’avoir diminué la vitesse max de déplacement de 30m/s à 15m/s. En fait le **Frame Skipping** avançait trop la physique avant que le drone puisse reprendre la main.
- Les **paramètres**. Là c’est pareil, malheureusement, si on change la valeur d’epsilon, par exemple en la passant de 0.0008 à 0.0002, ça change

complètement la donne car sur 2000 epochs, ça fait moins d'actions choisies aléatoirement, et du coup, le drone a moins exploré, et il a pas appris à voler. Mais faut pas non plus que ce soit trop grand, sinon il va faire n'importe quoi et il va jamais utiliser ses connaissances.

Du coup,

12.1 Améliorer interface graphique et génération de la map

Concerne principalement le html/css/js, les fichiers avec des fonctions mains (dans /app), simulation.c pour les obstacles et .env.c pour gérer les collision.

- Faire un "mixte" des programmes principaux main de simu et droneTest: soit le drone est piloté par l'IA, soit si on appuie sur un touche c'est pilote manuel et ça redevient automatique après un court lapse de temps.
- Peut-être mettre un bouton pour "générer aléatoirement" une map et qui démarre une simulation (de test)
- Améliorer la gestion des obstacles / utilisateurs. La génération de la map est essentielle: pour choisir une bonne situation initiale pour commencer l'apprentissage, donc dans droneTrain.c, mais aussi pour la génération random au milieu de l'apprentissage en fonction des epochs donc l'avancement, et aussi pour le test avec l'interface graphique. Cela revient globalement à factoriser (dans un fichier generation.c ou obstacles.c par exemple) et clean les fonctions type: `getHumanPenalty`, `getObstaclePenalty`, `captureObstacles`, `handleCollision` pour que les paramètres et distances de sécurité et espacements soient cohérents
- Il faut aussi que les obstacles / utilisateurs soient: pas tous trop proche et empilés, pas trop sur les bords, il faut aussi tester les hitboxes (donc la marge de sécurité). Autrement, le drone peut se comporter parfaitement en optimisant le réseau, stabilisant son vol, puis faire une action pas bonne (ce qui peu arriver) et qui ne serait pas destructive en temps normal (comme un petit coup à gauche), mais qui si réalisée trop proche d'un obstacles, c'est le crash.
- rendre les utilisateurs mobiles (ils se déplacent)

12.2 Améliorer le modèle

- Faire varier les hyperparamètres, modifier la fonction récompense et essayer de dénicher les comportements extrêmes qui peuvent être néfastes pour améliorer les résultats.